Software Reengineering

Mids Notes | Chapter 1 se 4 + Khud karne wale 9 topics | Aasan Roman Urdu mein, examples ke saath

Zaroori baat: is subject ki slides ya book nahi mili, is liye yeh notes standard textbooks ki maloomat (Pressman, Sommerville, Fowler, Chikofsky and Cross) par based hain. Kuch jagah sir ki slides mein points ka naam ya tarteeb thori alag ho sakti hai, to paper se pehle apni slides se ek baar milaan kar lena. Har topic mein **asaan matlab**, **example** (peela box) aur **exam point** (hara box) hai.

Chapter 1: Legacy Software Systems

1.1 Legacy system kya hota hai?

Asaan matlab: Legacy system woh **purana software** hai jo abhi bhi chal raha hai aur business ke liye zaroori hai, magar purani technology par bana hone ki wajah se usay badalna, samajhna aur chalana **mushkil aur mehnga** ho gaya hai.

Definition (exam ke liye): A legacy system is an old software system that was developed using outdated technology, but still performs critical business functions and is hard to modify and maintain.

Example: Bank ka woh core system jo 30-40 saal pehle COBOL language mein mainframe computer par bana tha aur aaj bhi roz lakhon transactions karta hai. Ya sarkari record ka purana software jis ke banane walay retire ho chuke hain. Yeh 'purane' hain magar band nahi kiye ja sakte, kyunke un par poora business chal raha hai.

1.2 Legacy systems ki khasiyatein

- **Purane aur bare:** saalon se chal rahe, lakhon lines of code.
- **Business-critical:** band ho gaye to business ruk jata hai.
- **Documentation kam ya ghalat:** jo likha tha woh purana ho chuka, ya likha hi nahi gaya.
- **Original developers chale gaye:** system ka ilm unke saath chala gaya (knowledge loss).
- **Purani technology:** purani languages (COBOL, Fortran), purane databases aur hardware.
- **Changes mushkil aur mehnge:** ek jagah badlo to doosri jagah kuch toot jata hai.
- **Structure kharab:** baar baar patch lagne se code uljha hua (spaghetti code).

1.3 Legacy system itna lamba kyun chalta hai?

- **Replace karna risky:** naya system banane mein ghalti ho gayi to business ko bohat nuqsan.
- **Business rules andar chhupe hain:** kai saalon ki policies code mein hain, kisi ko puri tarah yaad nahi.
- **Naya banana mehnga:** budget aur waqt dono.
- **Chal raha hai to chalne do:** 'if it works, do not touch it' wali soch.

1.4 Legacy systems ke masle

Masla	Matlab	Example
Zyada maintenance cost	Budget ka bara hissa sirf chalane aur patch karne mein jata hai.	Chhota sa change karne mein hafton lagna.
Documentation nahi	Naye log system samajh nahi pate.	Naya developer poochta hai 'yeh function kya karta hai?' aur jawab dene wala koi nahi.
Skills ki kami	Purani technology jaanne wale log kam aur mehnge.	COBOL developer dhoondna mushkil.
Integration mushkil	Naye systems (web, mobile, cloud) se jurna aasan nahi.	Purane bank system ko mobile app se connect karna.

Masla	Matlab	Example
Hardware purana	Purani machines ke spare parts aur support khatam.	Mainframe ka hardware kharab ho to replacement nahi.
Security khatre	Purane software mein naye khatron ke mutabiq security nahi.	Purani login method jo aaj aasani se hack ho jaye.
Software erosion / decay	Waqt ke saath patches se structure bigarta hai aur quality girti hai.	Purani imarat jis mein jagah jagah jodh toar ki mussibat.

1.5 Legacy system ke hisse (components)

Legacy system sirf code nahi hota, kayi cheezon ka majmua hota hai:

Hissa	Matlab
System hardware	Mainframe ya purani machines jin par chalta hai.
Support software	Operating system, compilers, utilities.
Application software	Asal programs, aksar bohat se alag alag modules.
Application data	Database/files; aksar bohat bara, kabhi kabhi ghalat ya duplicate data.
Business processes	Woh kaam ke tareeqe jo is system ke zariye chalte hain.
Business policies and rules	Qanoon, company ke usool jo code ke andar chhupe hain.

1.6 Legacy system ka kya kiya jaye? (4 options)

Management ko faisla karna hota hai. Do cheezein dekhi jati hain: **business value** (system kitna zaroori hai) aur **technical quality** (system andar se kitna achha hai).

Option	Kab chuna jata hai	Example
Scrap (band karo)	Business value kam aur quality kharab.	Ab koi use nahi karta.
Maintain as it is	Quality achhi aur value achhi, thori maintenance kaafi.	Theek chalta hua system.
Reengineer	Value zyada magar quality kharab: andar se sudharo, kaam wohi rakho.	Zaroori magar uljha hua system.
Replace (naya banao)	Technology bilkul kaam ki nahi, ya reengineer se naya banana sasta.	Purana system ab kisi tarah chal nahi sakta.

Exam mein yaad rakho: Value zyada + quality kam = Reengineering sab se munasib. Yehi is subject ki buniyaad hai.

1.7 Legacy ko naye system se jodne ke do tareeqe

- **Wrapping (black-box):** purane system ko uske upar naya 'khol' (interface) laga kar use karna, andar kuch nahi chhedte. Jaise purani machine par naya display laga dena.
- **Migration / Reengineering (white-box):** andar ka code aur structure samajh kar sudharna ya naye platform par le jana.

Chapter 2: Software Reengineering

2.1 Software Reengineering kya hai?

Asaan matlab: Purane software ko **dobara tayyar karna** taake woh behtar, saaf aur aaj ke zamane ke qabil ho jaye, magar uska kaam (functionality) wohi rahe. Jaise purani imarat ko gira kar nayi banane ke bajaye **renovate** karna: bunyaad aur kamray wohi, magar andar se naya.

Definition: Software reengineering is the examination and alteration of an existing system to reconstitute it in a new form, and the subsequent implementation of that new form (Chikofsky and Cross).

Formula yaad rakho: Reengineering = Reverse Engineering + (Restructuring) + Forward Engineering.

Example: Purana Student Result system jo uljhe code mein hai: reengineering mein pehle us ka code parh kar samjha (reverse), phir code ko saaf kiya (restructure), phir naye framework mein dobara bana diya (forward). Student ko result dekhne ka tareeqa wohi, magar andar se system saaf, tez aur aasaan.

2.2 Reengineering kyun karte hain? (Objectives)

- **Maintainability barhana:** system ko samajhna aur badalna aasan ho.
- **Cost kam karna:** naye system se sasta, aur maintenance ka kharcha ghatay.
- **Risk kam karna:** zero se likhne ke muqable mein business knowledge mehfooz rehti hai.
- **Nayi technology apnana:** web, mobile, cloud ya naye database par le jana.
- **Documentation dobara banana:** jo kho gayi thi.
- **Quality aur reliability behtar karna.**

2.3 Reengineering vs Maintenance vs Naya system

Tareeqa	Matlab	Cost	Risk
Maintenance	Chhote chhote changes: bug fix, chhoti tabdeeli.	Kam	Kam
Reengineering	Poore system ya bare hisse ko andar se sudharna, functionality wohi.	Darmiyana	Darmiyana
Replacement (new development)	Zero se naya system.	Zyada	Zyada

2.4 Terminology (Chikofsky and Cross): bohat important

In alfaaz mein log aksar confuse hote hain. Is table ko yaad kar lo:

Term	Matlab	Example
Forward engineering	**Design se code** ki taraf: normal software banane ka tareeqa (requirements -> design -> code).	Naya app banana.
Reverse engineering	**Code se design** ki taraf: maujooda code/system ko analyse kar ke uska design, structure aur specification nikalna. Code badalta nahi, sirf samajha jata hai.	Purane code ko parh kar class diagram aur flow chart banana.
Redocumentation	Reverse engineering ka ek hissa: same level par nayi documentation banana (comments, flowcharts, data dictionary).	Purane code ki user manual dobara likhna.
Design recovery	Code aur domain knowledge se **design ki tafseel wapas** hasil karna.	Code se pata lagana ke 'yeh module fee calculate karta hai'.
Restructuring	Same abstraction level par **structure sudharna**, behavior wohi.	Uljhe code ko saaf code banana.
Reengineering	Pori tabdeeli: reverse + restructure + forward.	Purane system ko naye rup mein dhalna.

Exam mein yaad rakho: Reverse engineering mein **samjha** jata hai, badla nahi jata. Restructuring mein **structure badalta hai magar behavior nahi**. Reengineering in sab ka majmua hai.

2.5 Reengineering ke strategies

Strategy	Matlab	Faida	Nuqsan
Big-bang (ek saath)	Poore system ko ek hi dafa reengineer karke naya chala dena.	Jaldi poora naya system.	Bohat risky; agar masla aya to sab ruk jata hai.
Incremental (hissa hissa)	System ke chhote chhote hisse ek ek kar ke reengineer karna, purana aur naya kuch der saath chalte hain.	Risk kam, ghaltiyan jaldi pakri jati hain.	Purane aur naye ko jorne ka extra kaam.

2.6 Reengineering ke fayde aur nuqsan

	Details
Fayde	Sasta aur kam risky naye system ke muqable mein; business knowledge mehfooz; maintainability aur quality behtar; nayi technology ka raasta.
Nuqsan / limits	Bohat kharab design ho to sirf reengineering se hal nahi hota; bohat zyada waqt aur mehnat; purani architecture ki hadein poori tarah nahi mitti; staff ko purani aur nayi dono technology aani chahiye.

Chapter 3: Software Evolution and Maintenance

3.1 Software Maintenance kya hai?

Asaan matlab: Software deliver hone (release) ke **baad** usay theek aur kaam ka rakhne ke liye jo bhi kaam kiye jate hain. Software gari ki tarah hai: chalti hai to servicing, tyre badalna aur kabhi accessories lagana parta hai.

Definition: Software maintenance is the modification of a software product after delivery to correct faults, improve performance or other attributes, or adapt it to a changed environment (IEEE).

3.2 Maintenance ki 4 qismein

Qisam	Matlab	Example	Hissa (andaza)
Corrective	**Bug theek karna** jo users ne ya testing ne pakre.	App crash ho rahi thi, theek ki.	~ 20%
Adaptive	Nayi **environment** ke mutabiq badalna (naya OS, naya hardware, naya qanoon).	Naye Android version par app chalana. Tax rate badalne par software update.	~ 25%
Perfective	Naye features ya **performance behtar** karna jo user ne maange.	Search tez karna, naya report add karna.	~ 50%
Preventive	Aane wale masle se bachne ke liye **pehle se** structure sudharna.	Code saaf karna taake baad mein bug na aayen (refactoring).	~ 5%

Hissa wale numbers Lientz and Swanson ke mashhoor survey ke andaze hain; sir ke slides mein thore alag ho sakte hain.

Exam mein yaad rakho: Sab se zyada kaam **perfective** hota hai, aur preventive sab se kam. Aur isi preventive maintenance ka hissa reengineering/refactoring hai.

3.3 Software Evolution kya hai?

Asaan matlab: Software ki zindagi mein badalti zarooraton ke mutabiq **musalsal tabdeeli aur barhna**. Software ek baar ban kar mukammal nahi ho jata, jaise zinda cheez waqt ke saath badalta rehta hai. Evolution ka matlab hai us badlao ka poora silsila, jabke maintenance us badlao ke kaam hain.

3.4 Lehman ke Laws of Software Evolution (8)

Manny Lehman ne bade software systems ka muta'ala karke yeh usool diye. Easy examples ke saath:

Law	Matlab	Example
1. Continuing Change	Software ko zaroorat ke mutabiq **badalte rehna** parta hai, warna dheere dheere bekaar ho jata hai.	Purani app jo naye phones par na chale.
2. Increasing Complexity	Jaise jaise badlao hota hai, structure **uljhta** jata hai, jab tak koshish karke usay saaf na kiya jaye.	Patch par patch lagne se code spaghetti ban jata hai.
3. Self-Regulation	Software evolution ke apne **statistical patterns** hote hain (size, release rate), jo natural tor par bante hain.	Har release mein lagbhag ek jitni tabdeeliyan.
4. Conservation of Organisational Stability	Team ki **kaam karne ki raftar** lagbhag sthir rehti hai, chahe resources barha do.	Zyada developers daalne se kaam hamesha dugna nahi hota.
5. Conservation of Familiarity	Har release mein tabdeeli ek hadd tak hi ho sakti hai warna team aur users system ko **samajh nahi pate**.	Ek hi release mein poora interface badalne par users uljh jate hain.
6. Continuing Growth	Users ki nayi mangon ke liye software ki **functionality barhti rehti** hai.	WhatsApp mein roz naye features.
7. Declining Quality	Agar actively sudhara na jaye to software ki **quality girti jati hai**.	Purane patch, purane masle, performance girti hui.
8. Feedback System	Evolution ka process ek **feedback loop** hai: users ki feedback se badlao, aur badlao se nayi feedback.	App store reviews dekh kar update.

3.5 Maintenance mehnga kyun hota hai? (cost factors)

- **Team stability:** developers badalte rahein to system ka ilm kho jata hai.
- **Staff ki skills:** maintenance team ko aksar kam tajurba mil pata hai.
- **Program ki umar aur structure:** jitna purana aur uljha, utna mehnga.
- **Documentation ki kami** aur ghalat documentation.
- **Contractual responsibility:** banane wali aur maintain karne wali team alag ho to design ki parwah kam hoti hai.
- **Ripple effect:** aik change ka asar doosri jagahon par phailna, jis se aur ghaltiyan aati hain.

3.6 Maintenance process (aasan steps)

- **1. Change request:** user ya developer tabdeeli maangta hai.
- **2. Impact analysis:** dekhna ke change se kaun kaun si cheezein asar mein aayengi.
- **3. Planning aur design:** tabdeeli ka plan.
- **4. Implementation:** code badalna.
- **5. Regression testing:** check karna ke purani cheezein na tooti hon.
- **6. Release aur documentation update.**

3.7 Software ki zindagi ke stages (staged model)

Stage	Matlab
Initial development	Pehla version banana.
Evolution	Naye features aur badlaw aasani se hote hain.
Servicing	Sirf chhote bug fix; architecture ab kharab ho chuka.
Phase-out	Support kam, naye changes band; replace ki tayyari.

Stage	Matlab
Close-down	System band.

Reengineering kahan fit hota hai? Jab system **servicing** ke qareeb pahunch jata hai, reengineering usay wapas **evolution** ki halat mein la sakti hai.

Chapter 4: Software Reengineering Process Model

4.1 Reengineering process model kya hai?

Reengineering ek **sidha ek-do din ka kaam nahi**, balke ek **process** hai jis mein kayi kaam tarteeb se aur dohra kar hote hain. Pressman ke mutabiq yeh **6 activities ka cycle** hai. Yeh ek **chakr (cycle)** hai, line nahi: agar kisi step par masla ho to peechay ja kar dobara kiya ja sakta hai.

6 Activities (tarteeb yaad rakho): Inventory Analysis -> Document Restructuring -> Reverse Engineering -> Code Restructuring -> Data Restructuring -> Forward Engineering

Activity 1: Inventory Analysis

Company ke saare applications ki **list (inventory)** banana aur har ek ke bare mein zaroori maloomat jama karna: umar, size, business ki ahmiyat, maintenance ki cost, kitni baar badla gaya, kitna kharab chal raha hai. Phir faisla ke kis ko reengineer karna hai aur kis ko nahi, **candidates** ko priority dete hain (jo business ke liye zaroori hain aur jin ka maintenance sab se mehnga hai).

Example: Company ke 20 software hain. Inventory se pata chala ke 'Payroll' system 18 saal purana hai, roz use hota hai aur har mahine 40 bug aate hain. Isay reengineering ki list mein sab se upar rakha.

Activity 2: Document Restructuring

Purani, kam ya ghalat **documentation ko theek karna**. Teen options: (1) **Kuch nahi karna** agar system stable hai aur badalne wala nahi (har chiz ki documentation banana mehnga hota hai). (2) **Sirf us hisse ki documentation banana jo badal raha hai** (documentation 'jaise zaroorat ho'). (3) **Poori documentation dobara** agar system business-critical hai.

Activity 3: Reverse Engineering

Maujooda code se design nikalna. Code parh kar, tools ki madad se: data structures, architecture, process flows, aur program ka behavior samajhna. Naya kuch nahi banta, bas jo hai woh 'samajh aata hai' aur likha jata hai (design recovery aur redocumentation yahin hoti hai).

Reverse engineering ke abstraction levels (neeche se upar): Implementation (code), Structure (modules aur unke rishte), Function (har module ka kaam), Domain (business ki baat). Jitna upar, utni zyada samajh.

Activity 4: Code Restructuring

Sab se aam reengineering ka kaam. Program ka **code analyse kar ke uljhe, unstructured hisson ko structured form mein dobara likhna**, behavior badle baghair. Purane code mein GOTO, lambe functions, ulajhi nested conditions hoti hain. Restructuring ke baad code parhna aur maintain karna aasan ho jata hai. Nateeje review aur test kiye jate hain taake koi nayi ghalti na aaye. (Mazeed tafseel neeche 'code restructuring' topic mein.)

Activity 5: Data Restructuring

Jin systems mein data ka design kharab hai, wahan sirf code theek karna kaafi nahi. Data restructuring ek **full-scale reengineering** activity hai jo **data architecture** se shuru hoti hai: pehle maujooda data structures ko reverse engineer karna, phir data models, data names aur formats ko saaf aur standard karna. Purane flat files ya kharab database tables ko achhi database design mein badalna.

Example: Ek hi insaan ka naam 3 tables mein 3 alag shakalon mein (Ali Khan, A. Khan, Khan Ali). Data restructuring mein sab ko ek standard format aur ek jagah rakha jata hai.

Activity 6: Forward Engineering

Reverse engineering se jo design aur maloomat mili, unhein istemal kar ke **naya, behtar system banana**: kabhi naye platform par, naye framework ya architecture mein, aur kabhi naye features ke saath. Isay **reclamation** bhi kehte hain: purane software se qeemti cheezein (business logic) nikal kar naye mein daalna. Isme sirf purani functionality dobara nahi aati, naye business needs ki tarf bhi qadam hota hai.

4.2 Doosra model: Sommerville ka reengineering process

Sommerville ki kitaab mein reengineering ke yeh activities milte hain (tarteeb mein), jo upar wale model se milte julte hain:

- **Source code translation:** purani language se nayi language mein code badalna.
- **Reverse engineering:** program ko analyse kar ke samajhna aur documentation banana.
- **Program structure improvement:** control structure ko saaf karna.
- **Program modularisation:** related hisson ko ek jagah jama kar ke redundancy hatana.
- **Data reengineering:** data ko naye structure mein badalna aur saaf karna.

Exam mein yaad rakho: Agar sir ne Pressman wala model parhaya hai to 6 activities likho. Agar Sommerville wala, to 5 activities. Dono mein cheezein lagbhag wohi hain, bas naam aur tarteeb thori alag.

Khud karne wale 9 Topics

Topic 1: Software Restructuring

Asaan matlab: Software ke **structure ko behtar banana**, magar uska **behavior (bahar se kaam) bilkul nahi badalna**. Jaise kamray ke andar saman ki tarteeb badalna taake raasta saaf ho, kamray ki deewaren ya darwaze nahi badalte.

Restructuring ki qismein:

Qisam	Matlab	Example
Code restructuring	Program ke code ko saaf aur structured karna.	Lambe function ko chhote functions mein todna.
Data restructuring	Data ke design aur format ko saaf karna.	Duplicate tables hatana, naming standard karna.
Architecture restructuring	Poore system ke hisson ke rishte aur layers sudharna.	Tangled modules ko layers (UI, logic, database) mein alag karna.

Kyun karte hain: maintainability barhane ke liye; samajhna aasan banane ke liye; masle aur bugs kam karne ke liye; aage reengineering ya naye features ke liye tayyari.

Restructuring ki khas baat: yeh **same abstraction level par** hoti hai (code se code, design se design) aur **functionality nahi badalti**.

Topic 2: Restructuring ki Definition (exam ke liye)

Definition: Restructuring is the transformation of a software system from one representation form to another at the same relative abstraction level, while preserving the system's external behavior (functionality and semantics). (Chikofsky and Cross)

Aasan lafzon mein: Software ko andar se **nayi, behtar shakal** dena magar bahar se kaam wohi rakhna. Isay aksar **'code ko saaf karna'** samjha jata hai, jo ek had tak theek hai, magar restructuring code ke saath data aur architecture par bhi hoti hai.

Example: Do kamray ke ghar mein jab saman idhar udhar pada ho aur aap usay tarteeb se rakhte ho, to ghar wohi hai (functionality wohi), bas rehna aasan ho gaya. Yehi restructuring hai.

Topic 3: Code Restructuring ki Definition

Definition: Code restructuring is the technique of modifying the source code of a program to improve its structure and readability, without changing its external behavior. Typically it converts unstructured (spaghetti) code into well-structured, modular code.

Asaan matlab: Uljhe hue code ko aisi shakal dena jise parhna aur badalna aasan ho. Purane programs mein aksar GOTO, lambi functions aur ek ke andar ek 'if' ka jaal hota hai. Restructuring unhein 'sequence, selection, iteration' wali saaf structure mein badal deti hai.

Kya kya kiya jata hai: GOTO ko loops/if se badalna; lambay code ko functions mein todna; nested if ko saaf karna; magic numbers ko naam wale constants dena; dead (kabhi na chalne wala) code hatana; naam saaf karna.

Misal: pehle (unstructured/nested):

```
def fee(student):
    if student.type == "regular":
        if student.scholarship:
            if student.marks > 80:
                return 0
            else:
                return 5000
        else:
            return 10000
    else:
        return 12000
```

Baad mein (code restructuring ke baad, behavior wohi):

```
def fee(student):
    if student.type != "regular":
        return 12000
    if not student.scholarship:
        return 10000
    if student.marks > 80:
        return 0
    return 5000
```

Nateeja: nesting khatam, parhna aasan, wohi result. Yeh 'guard clauses' ka tareeqa hai.

Topic 4: Refactoring

Asaan matlab: Chhoti chhoti, **mehfooz tabdeeliyan** kar ke code ko andar se saaf aur behtar banana, baghair uske bahari behavior ko badle. Martin Fowler ne isay mashhoor kiya.

Definition (Fowler): Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior.

Refactoring ke usool: chhote chhote qadam; har qadam ke baad tests chalana (pehle se tests honi chahiye); naye features aur refactoring alag alag karna (do topiyan ek saath nahi); behavior badalna mana hai.

Refactoring vs Restructuring (farq):

Pehlu	Refactoring	Restructuring
Level	Code level par, chhote chhote methods/classes.	Wasee concept: code, data aur architecture sab.
Size	Choti, baar baar ki, aksar developer roz karta hai.	Bari, planned activity ho sakti hai.
Style	Agile/development ka hissa, tests ke saath.	Reengineering/maintenance ka hissa.
Behavior	Nahi badalta.	Nahi badalta.

Aam refactoring techniques:

Technique	Kya karta hai
Extract Method	Code ke tukre ko naya function bana do.
Extract Class	Bari class ko do alag classes mein todna.
Rename	Naam saaf aur matlab wala rakhna.
Move Method / Field	Method ko us class mein le jana jahan woh asal mein belong karta hai.
Inline Method	Bohat chhote bekar function ko seedha code mein daal dena.
Replace Conditional with Polymorphism	Lambi switch/if chain ki jagah subclasses use karna.
Pull Up Method	Do subclasses ka same method parent mein le jana.
Replace Magic Number with Constant	86400 ki jagah SECONDS_IN_DAY likhna.

Fayde: code samajhna aasan, bugs kam, naye features aasan, duplicate hat jata hai, design behtar. **Khatre:** bina tests ke refactor karna bug la sakta hai; zyada karne se waqt zaya.

Topic 5: Code Smell

Asaan matlab: Code mein aisi **nishani (smell)** jo batati hai ke yahan **design ya structure ka masla ho sakta hai**, chahe code abhi chal raha ho. Jaise khane se bu aaye to ho sakta hai kharab ho, jaanch karni parti hai. Smell ka matlab bug nahi, magar 'dhyan do aur refactor karo'.

Definition: A code smell is a surface indication in the source code that usually corresponds to a deeper problem in the system's design. (Kent Beck and Martin Fowler)

Smell	Matlab	Aam hal
Long Method	Bohat lamba function jo bohat kaam karta hai.	Extract Method.
Large Class	Class mein bohat zyada fields aur methods; kayi zimmedariyan.	Extract Class.
Duplicate Code	Same ya milta julta code kayi jagah.	Extract Method / Pull Up.
Long Parameter List	Function ko bohat zyada parameters.	Parameter ko object mein jama karna.
Feature Envy	Ek method doosri class ka data zyada use karta hai.	Move Method.
Shotgun Surgery	Ek chhote change ke liye kayi classes mein tabdeeli.	Related code ko ek jagah jama karna.
Divergent Change	Ek class kayi alag wajahon se badalti hai.	Extract Class (alag zimmedari).
Switch / long if chain	Type ke mutabiq lambi switch statements baar baar.	Polymorphism.
Dead Code	Aisa code jo kabhi chalta hi nahi.	Hata do.
Magic Numbers	Bina matlab ke numbers seedhe code mein (e.g. 0.17).	Named constant.
Data Clumps	Wohi 3-4 variables hamesha saath aate hain.	Ek class bana do.
Comments as deodorant	Ghalat code ko samjhane ke liye bohat zyada comments.	Code ko saaf karo, naam theek karo.

Smells ke 5 groups (Fowler/Jerzyk): Bloaters (bohat bare: Long Method, Large Class), **Object-Orientation Abusers** (OO ka ghalat istemal: Switch statements), **Change Preventers** (badlao mushkil: Shotgun Surgery, Divergent Change), **Dispensables** (bekar: Duplicate Code, Dead Code), **Couplers** (zyada jura hona: Feature Envy).

Exam mein yaad rakho: Smell = nishani, bug nahi. Smell dikhe to refactoring se hal karo.

Topic 6: Function Extraction (Extract Method / Function)

Asaan matlab: Bare function ke andar ka koi **tukra** alag **naye function** mein nikal dena aur us ka mutaaliq naam dena, aur purani jagah us naye function ko call kar dena. Yeh 'Long Method' smell ka sab se aam hal hai.

Steps: (1) wo tukra chuno jo aik kaam karta hai; (2) naya function banao jis ka naam batay ke kya karta hai; (3) tukra naye function mein copy karo; (4) jo variables use hote hain unhein parameters ya return value se do; (5) purani jagah function call lagao; (6) tests chalao.

Pehle (lamba function):

```
def print_invoice(order):
    # banner
    print("=== INVOICE ===")
    # total nikalna
    total = 0
    for item in order.items:
        total += item.price * item.qty
    # tax lagana
    total = total + total * 0.17
    print("Total:", total)
```

Baad mein (function extraction ke baad):

```
TAX_RATE = 0.17

def calculate_subtotal(order):
    return sum(i.price * i.qty for i in order.items)

def add_tax(amount):
    return amount + amount * TAX_RATE

def print_invoice(order):
    print("=== INVOICE ===")
    total = add_tax(calculate_subtotal(order))
    print("Total:", total)
```

Fayde: har function ek hi kaam karta hai (high cohesion); naam hi explanation ban jata hai (comments ki zaroorat kam); tukre kahin aur dobara use ho sakte hain (duplicate khatam); alag alag test karna aasan; parhna aur badalna (jaise tax rate) aasan.

Kab karein: function bohat lamba ho; code ka tukra comment se samjhana par raha ho; wohi code kayi jagah dohra raha ho.

Topic 7: Class Extraction (Extract Class)

Asaan matlab: Jab ek class **bohat saare kaam** karne lag jaye (Large Class smell), to us ke andar se **related fields aur methods ek nayi class** mein nikal dete hain, aur purani class us nayi class ko use karti hai. Yeh **Single Responsibility** usool par chalta hai: ek class, ek kaam.

Nishaniyan jin se pata chalta hai ke class extract karni chahiye: class bohat bari ho gayi; kuch fields aur methods sirf aik gharane ke hain; class kayi alag wajahon se badalti hai (Divergent Change); naam ke sath 'And' ya 'Manager' jaisa bohat sa kaam.

Pehle (ek class sab kuch kar rahi hai):

```
class Employee:
    name, salary
    street, city, postal_code
    def get_full_address(self): ...
    def calculate_pay(self): ...
```

Baad mein (Address ki nayi class nikal li):

```
class Address:
    street, city, postal_code
    def get_full_address(self): ...

class Employee:
    name, salary
    address = Address(...)
    def calculate_pay(self): ...
```

Steps: (1) class ki alag zimmedari pehchano; (2) nayi class banao; (3) related fields aur methods move karo; (4) purani class mein nayi class ka object rakho; (5) saare references update karo; (6) tests chalao.

Fayde: har class ek kaam karti hai; chhoti classes samajhna aur test karna aasan; reuse (Address kisi aur class mein bhi); coupling kam hoti hai.

Topic 8: Duplicate Code

Asaan matlab: Wohi ya milta julta code **kayi jagah dohraya hua**. Aam wajah: copy-paste. Yeh sab se mashhoor code smell hai.

Yeh bura kyun hai: bug aik jagah theek kiya to baqi jagah reh jata hai; change karna ho to har jagah karna parta hai (bhool ho sakti hai); code ka size aur maintenance cost barhta hai; inconsistent behavior.

Example: Teen jagah discount calculate karne ka wohi 5 line ka code copy kiya hua hai. Discount ka rate badalna ho to teeno jagah badalna parega, ek bhool gaye to alag alag jagah alag discount.

Code clones ki qismein:

Type	Matlab
Type 1	**Bilkul same** code (sirf spaces/comments ka farq).
Type 2	Structure same magar **naam ya values alag** (variable naam, constants).
Type 3	Copy ke baad **thori tabdeeli**: kuch lines add/delete/badli.
Type 4	Code alag likha hai magar **kaam bilkul wohi** (semantic clone).

Duplicate code kaise hatate hain: Extract Method (common code function mein), **Pull Up Method** (do subclasses ka same code parent mein), **Extract Class**, **Template Method** (steps same, sirf ek hissa alag), parameters se farq sambhalna. **Pehchan:** code review, aur tools jaise SonarQube, PMD CPD jo clones dhoondte hain.

Pehle:

```
def student_discount(price):
    return price - price * 0.10

def teacher_discount(price):
    return price - price * 0.10
```

Baad mein:

```
def apply_discount(price, rate=0.10):
    return price - price * rate
```

Topic 9: Code Reliability Improved

Asaan matlab: Reliability ka matlab hai software **bharose ke qabil** ho: baar baar, har haalat mein bina crash aur ghalti ke sahi kaam kare. Restructuring aur refactoring se code **saaf** hota hai, jis ki wajah se bugs chhupne ki jagah kam hoti hai aur reliability barhti hai.

Reliability kaise improve hoti hai:

Tareeqa	Kyun reliability barhati hai	Example
Code saaf aur chhota hone se	Parhna aasan, to ghalti jaldi nazar aati hai.	Long Method ko todna.

Tareeqa	Kyun reliability barhati hai	Example
Duplicate code hatane se	Ek bug ka ek hi theek; baqi jagah purana bug nahi rehta.	Discount ka ek function.
Dead aur bekar code hatane se	Kam code = kam jagah bug chhupne ke liye.	Kabhi na chalne wala function delete.
Complexity kam karne se	Kam raaste (paths) to kam hi galat nateeje.	Nested if ko guard clauses mein badalna.
Behtar error handling	Ghalat input ya error par app crash na ho.	try/except aur input check.
Tests aur refactoring ek saath	Har tabdeeli ke baad tests, to regression nahi aata.	Unit tests chalana.
Coupling kam, cohesion zyada	Ek jagah ka change doosri jagah nahi tora.	Extract Class.
Naming aur documentation	Galat fehmi se hone wali ghaltiyan kam.	Saaf variable naam.
Static analysis tools	Code chalaye baghair hi masle pakar lete hain.	Linters, SonarQube.

Reliability naapne ke tareeqe: defect density (kitne bugs per 1,000 lines), MTBF (do failures ke beech ka average waqt), failure rate, test coverage, aur cyclomatic complexity (kam ho to behtar).

Example: Pehle ek 200 line ka function tha jis mein tax ka hisaab 3 jagah alag alag tha, aur ek jagah ghalat. Function extraction se tax ka hisaab ek hi function mein aaya, jahan test likhi gayi. Ab tax hamesha sahi aata hai: yeh reliability barhna hai.

Exam mein yaad rakho: Restructuring/refactoring directly features nahi barhatay, magar **quality, maintainability aur reliability** barhate hain, kyunke saaf aur chhota code kam ghaltiyan paida karta hai.

Quick Revision Sheet

Topic	Yaad rakho
Legacy system	Purana, business-critical, mushkil se badalne wala software. Options: Scrap, Maintain, Reengineer, Replace.
Reengineering	Purane system ko examine aur alter kar ke nayi shakal dena, functionality wohi. = Reverse + Restructure + Forward.
Terminology	Forward (design->code), Reverse (code->design), Redocumentation, Design recovery, Restructuring (same level, same behavior).
Strategies	Big-bang vs Incremental.
Maintenance 4 types	Corrective, Adaptive, Perfective, Preventive.
Evolution	Lehman ke 8 laws: Continuing change, Increasing complexity, Self-regulation, Org. stability, Familiarity, Continuing growth, Declining quality, Feedback system.
Process model (Pressman)	Inventory analysis, Document restructuring, Reverse engineering, Code restructuring, Data restructuring, Forward engineering.
Restructuring	Code, Data, Architecture; behavior nahi badalta.
Refactoring	Chhote mehfooz qadam, tests ke saath; behavior wohi.
Code smell	Design problem ki nishani: Long Method, Large Class, Duplicate Code, Feature Envy, Shotgun Surgery.
Extract Method / Class	Lambe function/class ko todna; Single Responsibility.
Duplicate code	4 clone types; Extract Method, Pull Up, Template Method.
Reliability	Saaf code + kam duplicate + tests + error handling = kam bugs, zyada bharosa.

Mids ke possible sawal: (1) Legacy system define karein, khasiyatein batayein. (2) Reengineering kya hai, kyun zaroori, aur maintenance se farq. (3) Forward/reverse engineering aur restructuring mein farq. (4) Maintenance ki qismein example ke saath. (5) Lehman ke laws. (6) Reengineering process model ki 6 activities. (7) Refactoring aur code smells, 5 smells batayein. (8) Extract Method aur Extract Class example ke saath. (9) Duplicate code kya hai, kaise hatate hain. (10) Code reliability kaise improve hoti hai.

Allah kare mids behtareen jayein, Moto! Agar sir ki slides mil jayein to bhej dena, mein notes un ke mutabiq milaan kar dunga.